

CSA1717 - Assessment Tool 2 – Artificial Intelligence

Name: Monika M

Reg.No.: 192521435

1. BACKTRACKING SEARCH FOR DOCTOR SHIFT SCHEDULING

1.1 Problem Definition

A hospital needs to assign three doctors (**D1, D2, and D3**) to three different shifts (**Morning, Afternoon, and Night**) while satisfying the following constraints:

- D1 cannot be assigned to the Night shift.
- D3 cannot be assigned to the Morning shift.
- Each shift can have only one doctor.
- D2 must be scheduled before D3 (Morning → Afternoon → Night).
- Every doctor must be assigned exactly one shift.

This scheduling problem is treated as a **Constraint Satisfaction Problem (CSP)** and solved using the **Backtracking Search algorithm**.

1.2 Constraint Analysis

Unary Constraints

- $D1 \neq \text{Night}$
- $D3 \neq \text{Morning}$

Binary Constraint

- $\text{Shift}(D2) < \text{Shift}(D3)$

Global Constraint

- Each shift must be assigned to only one doctor.

These constraints reduce the possible assignments and help the backtracking algorithm eliminate invalid solutions efficiently.

1.3 Backtracking Search Process

Step 1: Assign Shift to D1

Available shifts: {Morning, Afternoon}

- Assign **D1** → **Morning** ✓ Valid

Step 2: Assign Shift to D2

Available shifts: {Morning, Afternoon, Night}

- Morning ✗ Already assigned to D1
- Afternoon ✓ Valid

Step 3: Assign Shift to D3

Available shifts: {Afternoon, Night}

- Afternoon ✗ Already assigned to D2
- Night ✓ Valid

The final assignment satisfies:

- D2 works before D3.
- D1 is not assigned Night.
- D3 is not assigned Morning.
- Every shift is assigned to exactly one doctor.

1.4 Final Schedule

Doctor	Assigned Shift
--------	----------------

D1	Morning
D2	Afternoon
D3	Night

The Backtracking Search algorithm successfully finds a valid schedule while satisfying all constraints.

2. ROBOT GRID NAVIGATION USING BREADTH FIRST SEARCH (BFS)

2.1 Problem Definition

A robot must travel from the **Start (S)** position to the **Goal (G)** in a 5×5 grid containing obstacles.

Movement is allowed only in four directions:

- Up
- Down
- Left
- Right

Each movement has a cost of **1**, and the objective is to find the shortest possible path.

2.2 Constraint Analysis

The robot must satisfy the following conditions:

- Diagonal movement is not allowed.
- Obstacle cells cannot be crossed.
- Every move has equal cost.
- BFS explores nodes level by level to guarantee the shortest path.
- Manhattan Distance can estimate the distance to the goal but is not used for node expansion in BFS.

2.3 BFS Search Process

Step 1: Initialization

- Queue = [Start]
- Visited = {Start}

Step 2: Expand Start Node

The robot examines all valid neighboring cells and adds them to the queue.

Step 3: Level-by-Level Exploration

Nodes are explored in **First-In, First-Out (FIFO)** order.

Each newly discovered valid node is:

- Marked as visited.
- Added to the queue.
- Connected to its parent for path reconstruction.

Step 4: Goal Reached

When the Goal node is found:

- BFS terminates.
- The shortest path is reconstructed using parent nodes.

2.4 Optimal Path and Cost

Example:

$S \rightarrow N1 \rightarrow N2 \rightarrow \dots \rightarrow G$

If the total number of moves is **k**, then

Total Cost = $k \times 1 = k$

Since every move has the same cost, BFS always guarantees the shortest path.

3. AUTONOMOUS RESCUE ROBOT USING UNIFORM COST SEARCH (UCS)

3.1 Problem Definition

An autonomous rescue robot must reach a target location while navigating through a partially known environment.

The robot may encounter:

- Free cells
- Obstacle cells
- Risky zones

Its objective is to reach the goal with the **minimum total travel cost**.

3.2 Constraint Analysis

The robot operates under the following conditions:

- Movement allowed only in four directions.
- Obstacle cells cannot be crossed.
- The robot can observe only adjacent cells.
- Normal movement cost = **1**
- Risky zone movement cost = **3**
- The robot must continuously update its knowledge of the environment.
- The robot should always choose the least-cost route.

These constraints require the robot to make intelligent decisions while navigating.

3.3 Online Uniform Cost Search Process

Step 1

Initialize the map with only the Start position.

Step 2

Sense all adjacent cells and classify them as:

- Free
- Obstacle
- Risky

Step 3

Run **Uniform Cost Search (UCS)** using the currently known map.

The priority queue always selects the node with the lowest cumulative cost.

Step 4

Move to the selected lowest-cost neighboring cell.

Step 5

Sense the surroundings again from the new position.

Step 6

If new obstacles or risky zones are discovered:

- Update the internal map.
- Re-run UCS from the current location.

Step 7

Repeat the process until the Goal is reached.

This allows the robot to perform **online search**, where planning and execution occur simultaneously.

3.4 Application of Artificial Intelligence Concepts

Intelligent Agent

The rescue robot behaves as an intelligent agent because it:

- Perceives its surroundings.
- Maintains an internal map.
- Makes autonomous decisions.
- Chooses actions that minimize travel cost.

Rational Behavior

The robot acts rationally by:

- Avoiding obstacles.
- Reducing movement and risk costs.
- Updating its plan whenever new information becomes available.
- Selecting the least-cost path to the destination.

Environment Characteristics

Property	Description
Observability	Partially Observable
Environment	Dynamic

Property	Description
Decision Process	Sequential
Knowledge	Initially Unknown

3.5 Justification of Using Uniform Cost Search

Uniform Cost Search is well suited for this problem because it:

- Finds the minimum-cost path.
- Handles different movement costs efficiently.
- Supports dynamic re-planning when the environment changes.
- Works effectively in partially observable environments.